

SS15 — Recursion: Exercises

Exercise 1 — Recursive Sum of an Array

Write a recursive function to calculate the sum of the elements in an integer array.

Example: For [3, 5, 7, 9], the function should return 24.

Exercise 2 — Recursive Fibonacci

Write a recursive function to calculate the n th Fibonacci number. Call the function to compute and print the 8th Fibonacci number.

Exercise 3 — Recursive Tribonacci

Write a recursive function to calculate the n th Tribonacci number, with the convention:

- $T_0 = 0, T_1 = 1, T_2 = 1$
- $T_n = T_{(n-1)} + T_{(n-2)} + T_{(n-3)}$, for $n \geq 3$

Call the function to compute and print $T(9)$.

Optional optimization: Add memoization to improve performance.

Exercise 4 — Recursive GCD with Memoization

Use recursion to find the greatest common divisor (GCD) of two numbers, storing intermediate results using a HashMap to avoid recomputation (memoization).

- Input: Two integers.
- Output: The GCD of the two numbers.

Exercise 5 — Fibonacci with Memoization

Write a program to calculate the n th Fibonacci number using recursion, storing previously computed values in a HashMap (memoization) to avoid recalculating them.

- Input: An integer n .
- Output: The n th Fibonacci number.

Exercise 6 — Reversing a Queue Using Recursion

Write a program that uses recursion to reverse the elements in a queue, without using any loop construct.

Exercise 7 — Recursive Sum of a Sequence

Write a program to calculate the sum of numbers from 1 to n ($1 + 2 + 3 + \dots + n$) using recursion.

Detailed requirements:

- Create a recursive function `int sum(int n)`.
- Base case: when $n = 1$, return 1.
- Display the sequence of numbers and the resulting sum.
- Add an iterative (loop-based) version for comparison (optional).

Example: Input: 10 → Output: Sum = 55

Exercise 8 — Abstract Class: Electronic Device Management System

Build a system to manage electronic devices using an Abstract Class.

1. Create an abstract class `ElectronicDevice`

- Attributes: `brand (String)`, `model (String)`, `price (double)`
- Abstract methods (must be implemented by subclasses): `double calculatePowerConsumption()` — calculates power consumption; `String getDeviceType()` — returns the device type
- Concrete method: `void displayInfo()` — displays general information about the device

2. Create two subclasses

a. Class `Smartphone` extends `ElectronicDevice`

- Additional attributes: `screenSize (double)`, `batteryCapacity (int)`
- Implement `calculatePowerConsumption()`: `batteryCapacity * 0.05`
- Implement `getDeviceType()`: `"Smartphone"`

b. Class `Laptop` extends `ElectronicDevice`

- Additional attributes: `ram (int)`, `screenSize (double)`
- Implement `calculatePowerConsumption()`: `ram * 8.5`
- Implement `getDeviceType()`: `"Laptop"`

3. Requirements in the Main class

- Create a `List<ElectronicDevice>`
- Add at least 3 devices (2 Smartphones, 1 Laptop)
- Iterate through the list and call `displayInfo()` for each device
- Calculate the total power consumption of all devices
- Find the device with the highest price

Advanced requirements (encouraged):

- Add validation (`price > 0`, `RAM > 0`, etc.)
- Add a `Tablet` class that also extends `ElectronicDevice`

Sample output:

```
=== DEVICE INFO ===  
Brand: Samsung, Model: S23, Price: 25000.0  
Device Type: Smartphone  
Power Consumption: 450.0 W  
-----  
...  
TOTAL POWER CONSUMPTION: XXXX W  
Most expensive device: ...
```